DSC 40B - Quiz 03

Name: Student ID:

About the quizzes:

- Quizzes in DSC 40B are *optional* and graded pass/fail.
- A score of 70% or higher earns a "pass" and 1.5 credits toward your final grade.
- If you don't pass, no credits are earned, but it doesn't hurt your grade.
- You have 30 minutes to complete the quiz.
- Unfortunately, we can't answer clarifying questions during the quiz. If you think a question has a bug or is unclear, please let us know in a private post on Campuswire after the quiz, and we'll take it into account when grading.

Problem 1.

Consider the implementation of `binary_search` from lecture, reproduced below:

```
import math
def binary_search(arr, t, start, stop):
    """
    Searches arr[start:stop] for t.
    Assumes arr is sorted.
    """
    if stop - start <= 0:
        return None
    middle = math.floor((start + stop)/2)
    if arr[middle] == t:
        return middle
    elif arr[middle] > t:
        return binary_search(arr, t, start, middle)
    else:
        return binary_search(arr, t, middle+1, stop)
```

Suppose `arr` is the array below, and that `binary_search(arr, 30, 0, len(arr))` is called.

```
[2, 5, 9, 14, 20, 27, 33, 41, 48, 56, 63, 70]
```

How many calls to `binary_search` are made in total, including the first call?

◯ 2 ◯ 3 ◯ 4 ◯ 5 ◯ 12

Problem 2.

Consider the version of `binary_search` below. It is the same as the code from lecture, except that a `print` statement has been added which prints the value of `start` at the beginning of each call.

```
import math
def binary_search(arr, t, start, stop):
    print(start) # <-- added print statement
    if stop - start <= 0:
        return None
    middle = math.floor((start + stop)/2)
    if arr[middle] == t:
        return middle
    elif arr[middle] > t:
        return binary_search(arr, t, start, middle)
    else:
        return binary_search(arr, t, middle+1, stop)
```

Suppose `binary_search` is called on a sorted array with `start = 0` and `stop = len(arr)`. True or False: the numbers printed are never decreasing. That is, if x is printed before y, then $x \leq y$.

◯ True

◯ False

Problem 3.

The function below claims to count the number of times that `t` appears in `arr[start:stop]`.

```
import math
def count(arr, t, start, stop):
    if stop - start == 0:
        return 0
    if stop - start == 1:
        if arr[start] == t:
            return 1
        else:
            return 0
    middle = math.floor((start + stop) / 2)
    return (
        count(arr, t, start, middle)
        +
        count(arr, t, middle, stop)
    )
```

Suppose it is called with `start = 0` and `stop = len(arr)`. Will it always return the correct count?

◯ Yes: it will always return the correct count.

◯ No: it may recurse infinitely.

◯ No: it may try to access the array at an invalid index.

◯ No: it will run without error, but the count it returns may be wrong.

Problem 4.

Which recurrence describes the time taken by `boo` below on an input of size n, when $n > 1$?

```
import math
def boo(n):
    if n <= 1:
        return
    boo(n / 3)
    for j in range(math.floor(n)):
        print(j)
    boo(n / 3)
```

◯ $T(n) = T(n/3) + \Theta(n)$

◯ $T(n) = 2T(n/3) + \Theta(n)$

◯ $T(n) = 2T(n/3) + \Theta(1)$

◯ $T(n) = T(2n/3) + \Theta(n)$

Problem 5.

What is the solution of the recurrence below?

$$T(n) = \begin{cases} 3T(n/3) + \Theta(n), & n > 1 \\ \Theta(1), & n = 1 \end{cases}$$

◯ $\Theta(\log n)$ ◯ $\Theta(n)$ ◯ $\Theta(n \log n)$ ◯ $\Theta(n^2)$ ◯ $\Theta(n^3)$

Problem 6.

Suppose you are sorting an array using the in-place `selection_sort` from lecture. After some number of iterations of its outer loop, the array looks like this:

```
[1, 3, 4, 4, 6, 9, 7, 8, 10]
```

What is the **largest** number of iterations of the outer loop that could have run so far?

◯ 3 ◯ 4 ◯ 5 ◯ 6 ◯ 8

Problem 7.

In lecture, we modified `selection_sort` so that it finds the kth order statistic (the kth smallest element) of an array. The code is below, along with the `find_minimum` function that it uses.

```
def select_k(arr, k):
    """Find kth order statistic."""
    n = len(arr)
    if n <= 1:
        return
    for barrier_ix in range(k):
        # find index of min in arr[start:]
        min_ix = find_minimum(arr, start=barrier_ix)
        #swap
        arr[barrier_ix], arr[min_ix] = (
                arr[min_ix], arr[barrier_ix]
        )
    return arr[k-1]

def find_minimum(arr, start):
    """Finds index of minimum. Assumes non-empty."""
    n = len(arr)
    min_value = arr[start]
    min_ix = start
    for i in range(start + 1, n):
        if arr[i] < min_value:
            min_value = arr[i]
            min_ix = i
    return min_ix
```

Let n be the length of `arr`, and suppose that `k` can be any integer from 1 to n.

a) What is the **best case** time complexity of `select_k`?

◯ $\Theta(1)$ ◯ $\Theta(\log n)$ ◯ $\Theta(n)$ ◯ $\Theta(n \log n)$ ◯ $\Theta(n^2)$

b) What is the **worst case** time complexity of `select_k`?

◯ $\Theta(1)$ ◯ $\Theta(\log n)$ ◯ $\Theta(n)$ ◯ $\Theta(n \log n)$ ◯ $\Theta(n^2)$

Problem 8.

Consider the `mergesort` function from lecture, shown below. Remember that `merge` takes $\Theta(n)$ time on arrays with n elements in total.

```
import math
def mergesort(arr):
    """Sort array in-place."""
    if len(arr) > 1:
        middle = math.floor(len(arr) / 2)
        left = arr[:middle]
        right = arr[middle:]
        mergesort(left)
        mergesort(right)
        merge(left, right, arr)
```

True or False: if the input array is already sorted, `mergesort` as written above takes $\Theta(n)$ time.

◯ True

◯ False

Problem 9.

Suppose `mergesort` is called on an array of size n. In total, how many calls to `merge` are made over all calls to `mergesort`? For simplicity, you may assume that n is a power of two.

◯ $\Theta(\log n)$

◯ $\Theta(n)$

◯ $\Theta(n \log n)$

◯ $\Theta(n^2)$

Problem 10.

In this problem, assume that A and B are **sorted** lists, each containing n numbers.

Consider the problem of finding the **biggest** difference between a number in A and a number in B. That is, we want the largest value of $|a - b|$, where a is any number in A and b is any number in B. For example, if $A = [1, 4, 9]$ and $B = [2, 3, 12]$, the biggest difference is $|1 - 12| = 11$.

What is the best possible (worst case) time complexity of any algorithm for this problem?

Hint: the lists are sorted. Try to think of the fastest algorithm you can that takes advantage of that fact.

◯ $\Theta(1)$ ◯ $\Theta(\log n)$ ◯ $\Theta(n)$ ◯ $\Theta(n \log n)$ ◯ $\Theta(n^2)$